汇编语言第五次作业

作业题目 5：递归调用

基本要求：

求斐波那契数列（Fibonacci Sequence）：F(n)=F(n-1)+F(n-2)，并且 F(0)=0,F(1)=1。

1、从键盘输入 n（1≤n≤100）；

2、根据 n 值输出第 n 个斐波那契数；

3、要求用递归实现；

4、过程中要求使用寄存器；

5、初始化你的栈空间，思考栈空间多大才不会发生 Stack Overflow；

6、单步跟踪程序运行，观察每一次调用或返回时栈空间的变化。

7、用 C 语言实现后察看反汇编代码并加注释；

第一部分：用汇编实现斐波那契数列

第1步：新建汇编文件

```
代码块
1   D:\myassembly
```

命名为：

```
代码块
1   fib.asm
```

第2步：编写汇编代码

本次使用递归调用，并用多个16位字保存计算结果，以支持 n=100。这里采用线性递归，避免普通递归的大量重复计算。

```
代码块
1   .MODEL SMALL
2   .STACK 1000H
3
```

```
.DATA
    prompt  DB 'Please input n (1-100): $'
    result  DB 0DH,0AH,'Fibonacci number: $'
    errorMsg DB 0DH,0AH,'Invalid input!$'
    newline DB 0DH,0AH,'$'

    ; 使用5个16位字保存大整数
    A    DW 5 DUP(0)
    B    DW 5 DUP(0)
    TEMP DW 5 DUP(0)
    WORK DW 5 DUP(0)

.CODE

START:
    MOV AX, @DATA
    MOV DS, AX

    ; 显示输入提示
    MOV AH, 09H
    LEA DX, prompt
    INT 21H

    ; DI保存输入的n
    XOR DI, DI

READ_INPUT:
    MOV AH, 01H
    INT 21H

    CMP AL, 0DH
    JE INPUT_DONE

    ; 检查是否为数字
    CMP AL, '0'
    JB INVALID
    CMP AL, '9'
    JA INVALID

    SUB AL, '0'
    XOR AH, AH
    MOV SI, AX

    ; DI = DI * 10 + 当前数字
    MOV AX, DI
    MOV BX, 10
    MUL BX
```

```
51      ADD AX, SI
52      MOV DI, AX
53
54      CMP DI, 100
55      JA INVALID
56
57      JMP READ_INPUT
58
59  INPUT_DONE:
60      CMP DI, 1
61      JB INVALID
62
63      ; 调用递归函数 FIB(n)
64      PUSH DI
65      CALL FIB
66      ADD SP, 2
67
68      ; 输出提示
69      MOV AH, 09H
70      LEA DX, result
71      INT 21H
72
73      ; 将结果复制到WORK
74      MOV SI, 0
75      MOV CX, 5
76
77  COPY_RESULT:
78      MOV AX, A[SI]
79      MOV WORK[SI], AX
80      ADD SI, 2
81      LOOP COPY_RESULT
82
83      ; 十进制转换
84      MOV CX, 0
85
86  CONVERT_LOOP:
87      MOV BX, 10
88      MOV SI, 8
89      XOR DX, DX
90
91  DIV_WORD:
92      MOV AX, WORK[SI]
93      DIV BX
94      MOV WORK[SI], AX
95      SUB SI, 2
96      CMP SI, 0
97      JGE DIV_WORD
```

```
 98
 99      ; 保存余数
100      PUSH DX
101      INC CX
102
103      ; 检查商是否为0
104      MOV AX, WORK[0]
105      OR AX, WORK[2]
106      OR AX, WORK[4]
107      OR AX, WORK[6]
108      OR AX, WORK[8]
109
110      JNE CONVERT_LOOP
111
112  PRINT_LOOP:
113      POP DX
114      ADD DL, '0'
115      MOV AH, 02H
116      INT 21H
117      LOOP PRINT_LOOP
118
119      MOV AH, 09H
120      LEA DX, newline
121      INT 21H
122
123      MOV AX, 4C00H
124      INT 21H
125
126  INVALID:
127      MOV AH, 09H
128      LEA DX, errorMsg
129      INT 21H
130
131      MOV AX, 4C01H
132      INT 21H
133
134
135  ; ================================
136  ; 递归子程序 FIB
137  ; 参数: [BP+4] = n
138  ; 结果保存在数组A中
139  ; ================================
140
141  FIB PROC NEAR
142
143      PUSH BP
144      MOV BP, SP
```

```
145
146       ; 保存寄存器
147       PUSH AX
148       PUSH BX
149       PUSH CX
150       PUSH DX
151       PUSH SI
152       PUSH DI
153
154       MOV AX, [BP+4]
155
156       CMP AX, 0
157       JNE RECURSIVE_CALL
158
159       ; 基本情况: F(0)=0
160       MOV A[0], 0
161       MOV A[2], 0
162       MOV A[4], 0
163       MOV A[6], 0
164       MOV A[8], 0
165
166       ; B保存F(1)=1
167       MOV B[0], 1
168       MOV B[2], 0
169       MOV B[4], 0
170       MOV B[6], 0
171       MOV B[8], 0
172
173       JMP FIB_EXIT
174
175   RECURSIVE_CALL:
176
177       ; 递归调用FIB(n-1)
178       DEC AX
179       PUSH AX
180       CALL FIB
181       ADD SP, 2
182
183       ; 保存原来的A到TEMP
184       MOV SI, 0
185       MOV CX, 5
186
187   SAVE_A:
188       MOV AX, A[SI]
189       MOV TEMP[SI], AX
190       ADD SI, 2
191       LOOP SAVE_A
```

```
192
193         ; A = B
194         MOV SI, 0
195         MOV CX, 5
196
197     COPY_B:
198         MOV AX, B[SI]
199         MOV A[SI], AX
200         ADD SI, 2
201         LOOP COPY_B
202
203         ; B = B + TEMP
204         MOV SI, 0
205         MOV CX, 5
206         XOR DX, DX
207
208     ADD_WORDS:
209         MOV AX, B[SI]
210         ADD AX, TEMP[SI]
211         ADC AX, DX
212
213         MOV B[SI], AX
214
215         MOV DX, 0
216         ADC DX, 0
217
218         ADD SI, 2
219         LOOP ADD_WORDS
220
221     FIB_EXIT:
222
223         ; 恢复寄存器
224         POP DI
225         POP SI
226         POP DX
227         POP CX
228         POP BX
229         POP AX
230         POP BP
231
232         RET
233
234     FIB ENDP
235
236     END START
```

第3步：编译与运行

打开之前配置好的 DOSBox，依次输入：

1. 汇编

```
masm fib.asm
```

```
D:\>masm fib.asm
Microsoft (R) Macro Assembler Version 5.00
Copyright (C) Microsoft Corp 1981-1985, 1987.  All rights reserved.

Object filename [fib.OBJ]:
Source listing  [NUL.LST]:
Cross-reference [NUL.CRF]:
fib.asm(39): error A2053: Jump out of range by 15 byte(s)
fib.asm(41): error A2053: Jump out of range by 11 byte(s)

  51564 + 464980 Bytes symbol space free

      0 Warning Errors
      2 Severe  Errors
```

有错误，定位原因可能是MASM 5.0 的条件跳转指令只能跳转较短的距离，而我写的 JB INVALID、JA INVALID 跳得太远了。

修正后的fib.asm如下

```

.MODEL SMALL
.STACK 1000H

.DATA
    prompt   DB 'Please input n (1-100): $'
    result   DB 0DH,0AH,'Fibonacci number: $'
    errorMsg DB 0DH,0AH,'Invalid input!$'
    newline  DB 0DH,0AH,'$'

    ; 5个16位字组成80位整数
    A    DW 5 DUP(0)
    B    DW 5 DUP(0)
    TEMP DW 5 DUP(0)
    WORK DW 5 DUP(0)

```

```
.CODE

START:
    MOV AX, @DATA
    MOV DS, AX

    ; 显示输入提示
    MOV AH, 09H
    LEA DX, prompt
    INT 21H

    XOR DI, DI

; =========================
; 输入1-100之间的整数
; =========================

READ_INPUT:
    MOV AH, 01H
    INT 21H

    CMP AL, 0DH
    JNE CHECK_INPUT
    JMP INPUT_DONE

CHECK_INPUT:
    CMP AL, '0'
    JAE CHECK_UPPER
    JMP INVALID

CHECK_UPPER:
    CMP AL, '9'
    JBE DIGIT_OK
    JMP INVALID

DIGIT_OK:
    SUB AL, '0'
    XOR AH, AH
    MOV SI, AX

    ; 防止输入溢出
    CMP DI, 10
    JBE INPUT_SAFE
    JMP INVALID

INPUT_SAFE:
    ; DI = DI * 10 + 当前数字
```

```
64      MOV AX, DI
65      MOV BX, 10
66      MUL BX
67      ADD AX, SI
68      MOV DI, AX
69
70      CMP DI, 100
71      JBE READ_INPUT
72      JMP INVALID
73
74  INPUT_DONE:
75      CMP DI, 1
76      JAE INPUT_VALID
77      JMP INVALID
78
79  INPUT_VALID:
80      ; 调用递归函数
81      PUSH DI
82      CALL FIB
83      ADD SP, 2
84
85      ; 输出提示
86      MOV AH, 09H
87      LEA DX, result
88      INT 21H
89
90      ; 复制A到WORK
91      MOV SI, 0
92      MOV CX, 5
93
94  COPY_RESULT:
95      MOV AX, A[SI]
96      MOV WORK[SI], AX
97      ADD SI, 2
98      LOOP COPY_RESULT
99
100     ; =========================
101     ; 80位整数转换成十进制
102     ; =========================
103
104     MOV CX, 0
105
106 CONVERT_LOOP:
107     MOV BX, 10
108     MOV SI, 8
109     XOR DX, DX
110
```

```
111  DIV_WORD:
112      MOV AX, WORK[SI]
113      DIV BX
114      MOV WORK[SI], AX
115
116      SUB SI, 2
117      CMP SI, 0
118      JGE DIV_WORD
119
120      ; 保存余数
121      PUSH DX
122      INC CX
123
124      ; 检查商是否为0
125      MOV AX, WORK[0]
126      OR AX, WORK[2]
127      OR AX, WORK[4]
128      OR AX, WORK[6]
129      OR AX, WORK[8]
130
131      JNE CONVERT_LOOP
132
133  PRINT_LOOP:
134      POP DX
135      ADD DL, '0'
136      MOV AH, 02H
137      INT 21H
138      LOOP PRINT_LOOP
139
140      MOV AH, 09H
141      LEA DX, newline
142      INT 21H
143
144      MOV AX, 4C00H
145      INT 21H
146
147  ; =========================
148  ; 输入错误处理
149  ; =========================
150
151  INVALID:
152      MOV AH, 09H
153      LEA DX, errorMsg
154      INT 21H
155
156      MOV AX, 4C01H
157      INT 21H
```

```
158
159 ; =========================
160 ; 递归函数 FIB
161 ; 参数: [BP+4] = n
162 ; A保存F(n)
163 ; B保存F(n+1)
164 ; =========================
165
166 FIB PROC NEAR
167
168     PUSH BP
169     MOV BP, SP
170
171     ; 保存寄存器
172     PUSH AX
173     PUSH BX
174     PUSH CX
175     PUSH DX
176     PUSH SI
177     PUSH DI
178
179     MOV AX, [BP+4]
180
181     CMP AX, 0
182     JNE RECURSIVE_CALL
183
184     ; 基本情况: F(0)=0
185     MOV A[0], 0
186     MOV A[2], 0
187     MOV A[4], 0
188     MOV A[6], 0
189     MOV A[8], 0
190
191     ; B=F(1)=1
192     MOV B[0], 1
193     MOV B[2], 0
194     MOV B[4], 0
195     MOV B[6], 0
196     MOV B[8], 0
197
198     JMP FIB_EXIT
199
200 RECURSIVE_CALL:
201
202     ; 递归调用FIB(n-1)
203     DEC AX
204     PUSH AX
```

```
205       CALL FIB
206       ADD SP, 2
207
208       ; 保存A到TEMP
209       MOV SI, 0
210       MOV CX, 5
211
212   SAVE_A:
213       MOV AX, A[SI]
214       MOV TEMP[SI], AX
215       ADD SI, 2
216       LOOP SAVE_A
217
218       ; A=B
219       MOV SI, 0
220       MOV CX, 5
221
222   COPY_B:
223       MOV AX, B[SI]
224       MOV A[SI], AX
225       ADD SI, 2
226       LOOP COPY_B
227
228       ; B=B+TEMP
229       ; 使用ADC逐字传递进位
230       MOV SI, 0
231       MOV CX, 5
232       CLC
233
234   ADD_WORDS:
235       MOV AX, B[SI]
236       ADC AX, TEMP[SI]
237       MOV B[SI], AX
238
239       INC SI
240       INC SI
241       LOOP ADD_WORDS
242
243   FIB_EXIT:
244
245       ; 恢复寄存器
246       POP DI
247       POP SI
248       POP DX
249       POP CX
250       POP BX
251       POP AX
```

```
252       POP BP
253
254       RET
255
256   FIB ENDP
257
258   END START
```

编译运行

1. 汇编

代码块

```
1   masm fib.asm
```

```
D:\>masm fib.asm
Microsoft (R) Macro Assembler Version 5.00
Copyright (C) Microsoft Corp 1981-1985, 1987.  All rights reserved.

Object filename [fib.OBJ]:
Source listing  [NUL.LST]:
Cross-reference [NUL.CRF]:

  51564 + 464980 Bytes symbol space free

      0 Warning Errors
      0 Severe  Errors
```

2. 链接

代码块

```
1   link fib.obj
```

```
D:\>link fib.obj

Microsoft (R) Overlay Linker  Version 3.60
Copyright (C) Microsoft Corp 1983-1987.  All rights reserved.

Run File [FIB.EXE]:
List File [NUL.MAP]:
Libraries [.LIB]:
```

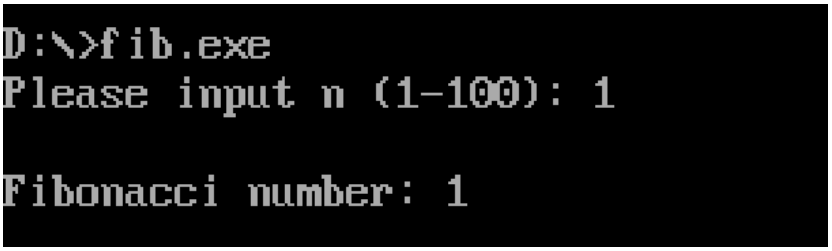
3. 运行

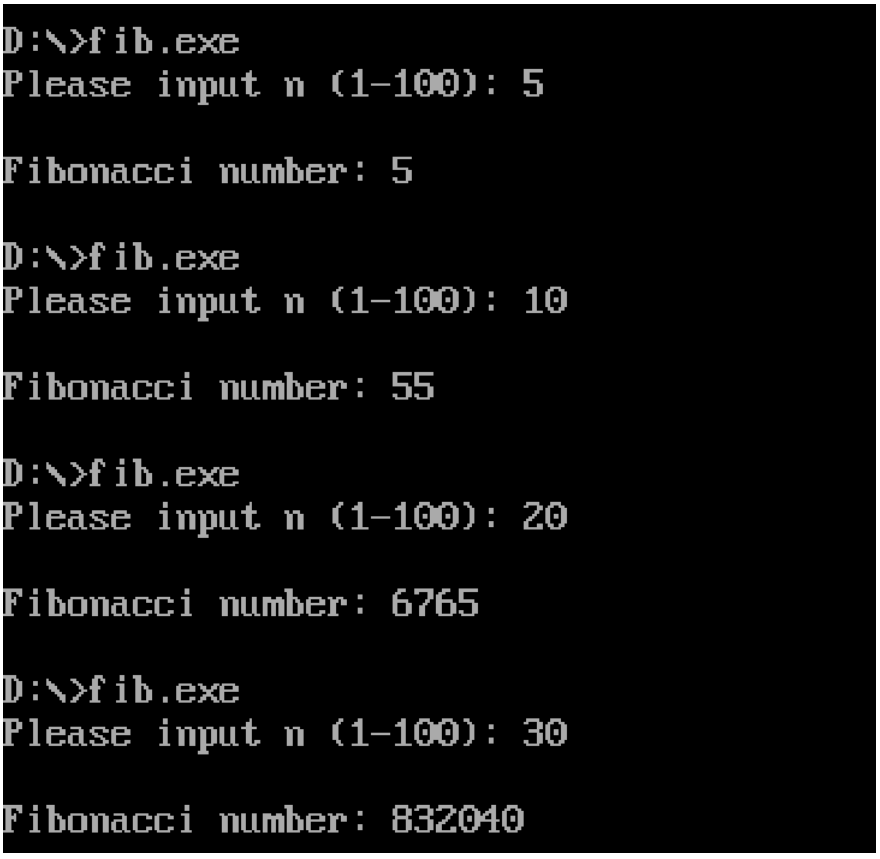
```
代码块ib.exe
```

第4步：测试程序

测试以下数据：

输入 n	预期结果 F(n)
1	1
5	5
10	55
20	6765
30	832040
50	12586269025
100	354224848179261915075





```
D:\>fib.exe
Please input n (1-100): 50

Fibonacci number: 12586269025

D:\>fib.exe
Please input n (1-100): 100

Fibonacci number: 354224848179261915075
```

第二部分：DEBUG 单步跟踪递归调用与堆栈变化

第1步：重新进入 DEBUG

输入：

```
代码块
1   debug fib.exe
```

然后输入：

```
代码块
1   r
```

把显示的 CS 和 IP 记下来。

```
D:\>debug fib.exe
-r
AX=FFFF BX=0000 CX=01D2 DX=0000 SP=1000 BP=0000 SI=0000 DI=0000
DS=072E ES=072E SS=075C CS=073E IP=0000 NV UP EI PL ZR NA PE NC
073E:0000 B85407            MOV     AX,0754
```

第2步：设置正确的断点

FIB 入口是：

```
代码块
1   073F:00CB
```

注意，DEBUG 的命令有区别：

- `g=00CB`：从 00CB 开始执行。
- `g 00CB`：从当前程序位置执行，在 00CB 设置断点。

如果重新进入 DEBUG 后 CS 仍然是 `073F`，输入：

```
代码块
1   g 00CB
```

程序就会从入口开始运行，遇到 FIB 函数时暂停。

第3步：输入测试数据

程序出现：

```
代码块
1   Please input n (1-100):
```

输入：

```
代码块
1   3
```

按回车。

此时应该停在 FIB 函数入口附近。

输入：

```
代码块
1   r
```

看到：

```
代码块
1   073F:00CB  PUSH BP
```

```
-g 00CB
Please input n (1-100): 3
AX=010D BX=000A CX=01DZ DX=0000 SP=0FFC BP=0000 SI=0003 DI=0003
DS=0754 ES=072E SS=075C CS=073E IP=00CB NV UP EI PL NZ NA PO NC
073E:00CB 55                PUSH    BP
```

第4步：观察堆栈

然后逐条输入：

代码块

```
t
```

执行以下指令：

代码块

```
PUSH BP
MOV BP,SP
PUSH AX
PUSH BX
PUSH CX
PUSH DX
PUSH SI
PUSH DI
```

每执行一条 PUSH，SP 减少2。

```
-t
AX=010D BX=000A CX=01D2 DX=0000 SP=0FFA BP=0FFA SI=0003 DI=0003
DS=0754 ES=072E SS=075C CS=073E IP=00CE NV UP EI PL NZ NA PO NC
073E:00CE 50                PUSH    AX
-t
AX=010D BX=000A CX=01D2 DX=0000 SP=0FF8 BP=0FFA SI=0003 DI=0003
DS=0754 ES=072E SS=075C CS=073E IP=00CF NV UP EI PL NZ NA PO NC
073E:00CF 53                PUSH    BX
-t
AX=010D BX=000A CX=01D2 DX=0000 SP=0FF6 BP=0FFA SI=0003 DI=0003
DS=0754 ES=072E SS=075C CS=073E IP=00D0 NV UP EI PL NZ NA PO NC
073E:00D0 51                PUSH    CX
-t
AX=010D BX=000A CX=01D2 DX=0000 SP=0FF4 BP=0FFA SI=0003 DI=0003
DS=0754 ES=072E SS=075C CS=073E IP=00D1 NV UP EI PL NZ NA PO NC
073E:00D1 52                PUSH    DX
-t
AX=010D BX=000A CX=01D2 DX=0000 SP=0FF2 BP=0FFA SI=0003 DI=0003
DS=0754 ES=072E SS=075C CS=073E IP=00D2 NV UP EI PL NZ NA PO NC
073E:00D2 56                PUSH    SI
-t
AX=010D BX=000A CX=01D2 DX=0000 SP=0FF0 BP=0FFA SI=0003 DI=0003
DS=0754 ES=072E SS=075C CS=073E IP=00D3 NV UP EI PL NZ NA PO NC
073E:00D3 57                PUSH    DI
```

```
-t
AX=010D BX=000A CX=01D2 DX=0000 SP=0FEE BP=0FFA SI=0003 DI=0003
DS=0754 ES=072E SS=075C CS=073E IP=00D4 NV UP EI PL NZ NA PO NC
073E:00D4 8B4604            MOV     AX,[BP+04]                  SS:0FFE=0003
```

查看寄存器：

```
代码块

1   r
```

查看堆栈：

```
代码块

1   d ss:sp
```

```
-t
AX=010D BX=000A CX=01D2 DX=0000 SP=0FEE BP=0FFA SI=0003 DI=0003
DS=0754 ES=072E SS=075C CS=073E IP=00D4 NV UP EI PL NZ NA PO NC
073E:00D4 8B4604            MOV     AX,[BP+04]                  SS:0FFE=0003
-r
AX=010D BX=000A CX=01D2 DX=0000 SP=0FEE BP=0FFA SI=0003 DI=0003
DS=0754 ES=072E SS=075C CS=073E IP=00D4 NV UP EI PL NZ NA PO NC
073E:00D4 8B4604            MOV     AX,[BP+04]                  SS:0FFE=0003
-d ss:sp
075C:0FE0                           -                   03 00                ..
075C:0FF0  03 00 00 00 D2 01 0A 00-0D 01 00 00 55 00 03 00 ............U...
075C:1000  00 00 D2 01 00 00 20 0D-1A 10 20 01 20 0D 00 00 ......  ... . ...
075C:1010  00 00 00 00 D2 01 00 00-21 0D 2C 10 20 01 21 0D ........!.,. .!.
075C:1020  00 00 00 00 00 00 D2 01-00 00 22 0D 3E 10 20 01 ..........".>. .
075C:1030  22 0D 00 00 00 00 00 00-D2 01 00 00 23 0D 50 10 ".............#.P.
075C:1040  20 01 23 0D 00 00 00 00-00 00 D2 01 00 00 24 0D  .#...........$.
075C:1050  62 10 20 01 24 0D 00 00-00 00 00 00 D2 01 00 00 b. .$...........
075C:1060  25 0D 74 10 20 01 25 0D-00 00 00 00 00 00       %.t. .%.......
```

第5步：观察递归调用

找到：

代码块

```
PUSH AX
CALL FIB
```

执行 CALL 后：

- 返回地址压入堆栈。
- SP 减少2。
- IP 跳转到 FIB 函数入口。

再次记录 SP，就能看到递归调用造成的堆栈变化。

第三部分：C语言实现斐波那契数列

第1步：新建 C 语言文件

新建文件：

代码块

```
fib.c
```

代码块

```c
#include <stdio.h>

/* 使用128位整数保存斐波那契数 */
typedef unsigned __int128 BigInt;

/* 递归计算斐波那契数列 */
void fib(int n, BigInt *a, BigInt *b)
{
    if (n == 0)
    {
        *a = 0;
        *b = 1;
        return;
    }

    /* 递归调用 */
    fib(n - 1, a, b);

    /* 计算当前斐波那契数 */
    BigInt temp = *a;
    *a = *b;
    *b = temp + *b;
}

/* 输出128位整数 */
void printBigInt(BigInt num)
{
    char digits[40];
    int count = 0;

    if (num == 0)
    {
        printf("0");
        return;
    }

    while (num > 0)
    {
        digits[count++] = '0' + num % 10;
        num /= 10;
    }

    while (count > 0)
    {
        putchar(digits[--count]);
    }
}
```

```
48
49  int main(void)
50  {
51      int n;
52      BigInt a = 0, b = 1;
53
54      printf("Please input n (1-100): ");
55
56      scanf("%d", &n);
57
58      if (n < 1 || n > 100)
59      {
60          printf("Invalid input!\n");
61          return 1;
62      }
63
64      fib(n, &a, &b);
65
66      printf("Fibonacci number: ");
67      printBigInt(a);
68      printf("\n");
69
70      return 0;
71  }
```

这里使用 `unsigned __int128`，是为了让 C 程序也能够正确保存 F(100)，而不是像普通64位整数那样发生溢出。

第2步：编译并运行

打开 Windows CMD，输入：

代码块

```
1  cd /d D:\myassembly
```

编译：

代码块

```
1  gcc fib.c -o fib_c.exe
```

运行：

```
fib_c.exe
```

第3步：测试程序

依次测试：

输入	预期输出
5	5
10	55
20	6765
30	832040
50	12586269025
100	354224848179261915075

```
D:\myassembly>gcc fib.c -o fib_c.exe

D:\myassembly>fib_c.exe
Please input n (1-100): 5
Fibonacci number: 5

D:\myassembly>fib_c.exe
Please input n (1-100): 10
Fibonacci number: 55

D:\myassembly>fib_c.exe
Please input n (1-100): 20
Fibonacci number: 6765

D:\myassembly>fib_c.exe
Please input n (1-100): 30
Fibonacci number: 832040

D:\myassembly>fib_c.exe
Please input n (1-100): 50
Fibonacci number: 12586269025

D:\myassembly>fib_c.exe
Please input n (1-100): 100
Fibonacci number: 354224848179261915075
```

第四部分：生成 GCC 汇编代码并添加中文注释

第1步：生成汇编代码

在刚才的 CMD 窗口输入：

```
代码块
1   gcc -S -masm=intel -O0 fib.c -o fib_gcc.s
```

参数含义：

参数	作用
-S	只生成汇编代码
-masm=intel	使用 Intel 汇编语法
-O0	关闭优化，便于观察递归过程
-o fib_gcc.s	指定生成文件名

成功后，在 `D:\myassembly` 中会出现：

```
代码块
1   fib_gcc.s
```

第2步：添加中文注释

保存在D:\myassembly\fib_gcc_annotated.s